

## Final Report

[Team Github Repo](#)

### Abstract

**Author: Marcus Francois**

This project addresses the balance of computational cost and accuracy in convolutional neural networks and vision transformers for image classification. To properly assess the trade-off, a custom built Convolutional Neural Network (CNN) was developed from scratch and tested against a pre-trained ResNet-18 using the CIFAR-10 dataset. The experiment was focused on if the proficiency of transfer learning is superior to the efficiency of a custom built CNN. The custom built CNN only had a best validation accuracy of 0.729 at epoch 13 without augmentation and 0.7996 at epoch 14 with it. Whereas, the ViT had a best validation accuracy of 0.9910.

### Introduction

**Author: Marcus Francois**

In the broader context of deep learning, image classification is an important way of evaluating if artificial intelligence can properly perceive and categorize visual information. Convolutional Neural Networks (CNNs) are the gold standard of modern deep learning for image processing and computer vision, but finding the balance between cost and accuracy can be a challenge. This problem is shown perfectly in this project by comparing a built from scratch CNN to a pre-trained ResNet-18 using the CIFAR-10 dataset. This comparison serves as a test to determine if the capabilities of transfer learning outweigh the lightweight efficiency of a custom built model. Understanding the pros and cons is very important for real world applications.

### Methods

**Author: Jennifer Durnall**

*Methodology for First CNN Model – Contributors: Jennifer D.*

For the baseline model, I implemented a convolutional neural network (CNN) from scratch using Pytorch. The objective of this model was to establish a performance baseline for image classification without relying on pretrained weights, while also developing a detailed understanding of how CNNs learn hierarchical visual features directly from data. The design and training process followed standard practices from Pytorch tutorials and commonly used CIFAR-10 baseline implementations. The first step was importing the required libraries. Pytorch was used for model construction, tensor operations, loss computation, and optimization. The CIFAR-10 dataset was loaded using torchvision, which also provided transformation utilities. Additional libraries included NumPy and Python's random module, matplotlib for visualization, tqdm for progress tracking, and sklearn.metrics for evaluation. A fixed random seed was applied across Python, NumPy, and Pytorch, including CUDA, to ensure consistent data splitting and training behavior across runs.<sup>1</sup>

The runtime environment was configured using: `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`. This allowed the model to utilize GPU acceleration when

---

<sup>1</sup> PyTorch Reproducibility Developer Notes - <https://docs.pytorch.org/docs/2.11/notes/randomness.html>

available, which is critical for efficiently performing convolution operations. The hyperparameters were selected based on standard CIFAR-10 training practices. The batch size was set to 128,<sup>2</sup> which is commonly used because it balances computational efficiency with stable gradient estimation. The learning rate was set to 0.001, which is the default value for the Adam optimizer and is widely used for stable convergence.<sup>3</sup> The model was trained for 15 epochs, which is sufficient for a small CNN to converge on CIFAR-10 without excessive overfitting. A validation split of 0.2 was used to follow a standard 80/20 training-validation ratio. The dataset was preprocessed using: `transforms.ToTensor()` `//// transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))`. The normalization values were chosen because they are used in the official PyTorch CIFAR-10 tutorial, shifting pixel values to be centered around zero and improving gradient stability.<sup>4</sup> The training dataset was split into training validation subsets using `random_split`. This allowed performance to be evaluated during training, helping detect overfitting early. Dataloaders were then created. The training DataLoader used `shuffle=True` to randomize data order each epoch, preventing the model from learning order-based patterns. Validation and test Dataloaders used `shuffle=False` since they were only used for evaluation. Before training, sample images were visualized to confirm that preprocessing and labeling were correct.

The CNN architecture was implemented as a custom class inheriting from `nn.Module`. The model consisted of three convolution blocks, each structured as: `Conv2d`  $\rightarrow$  `ReLU`  $\rightarrow$  `MaxPool2d`. The convolutional layers increased in depth from 3 input channels to 32, then to 64, then to 128.<sup>5</sup> This pattern follows standard CNN design principles where deeper layers learn increasingly abstract features. A kernel size of 3x3 was used for all convolutional layers,<sup>6</sup> as this is a widely adopted standard in modern CNN architectures due to its ability to capture local spatial features efficiently. Max pooling was applied after each convolutional layer to reduce spatial dimensions, lower computational cost, and increase the receptive field of deeper layers<sup>7</sup>. One challenge encountered during this stage was managing tensor dimensions. Since pooling reduces spatial size, the flattened feature vector had to match the expected input size of the fully connected layer. Debugging shape mismatches required careful tracking of spatial dimensions throughout the network.

After feature extraction, the output was flattened and passed through a fully connected classifier: `Linear`  $\rightarrow$  256  $\rightarrow$  `ReLU`  $\rightarrow$  `Dropout(0.3)`  $\rightarrow$  `Linear`  $\rightarrow$  10. The hidden layer size of 256 was chosen to provide sufficient capacity for combining extracted features without introducing excessive parameters. Dropout was set to 0.3, which falls within the commonly recommended

---

<sup>2</sup> PyTorch Training a Classifier Developer Notes -

[https://docs.pytorch.org/tutorials/beginner/blitz/cifar10\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html)

<sup>3</sup> PyTorch Adam Developer Notes - <https://docs.pytorch.org/docs/2.11/generated/torch.optim.Adam.html>

<sup>4</sup> PyTorch Training a Classifier Developer Notes -

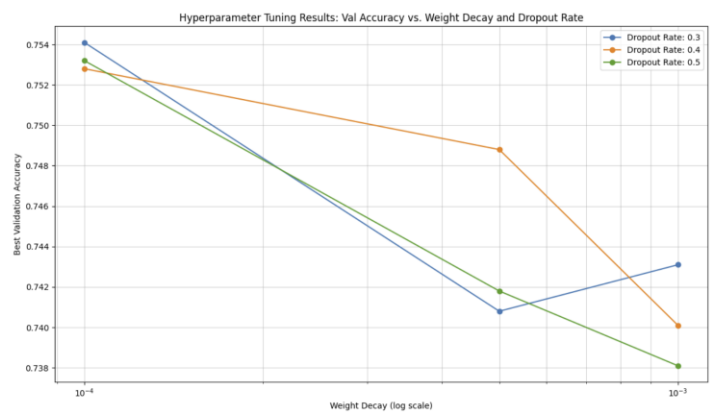
[https://docs.pytorch.org/tutorials/beginner/blitz/cifar10\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html)

<sup>5</sup> CS231 Deep Learning for Computer Vision - <https://cs231n.github.io/convolutional-networks/>

<sup>6</sup> Very Deep Convolutional Networks for Large-Scale Image Recognition - <https://arxiv.org/abs/1409.1556>

<sup>7</sup> CS231 Deep Learning for Computer Vision - <https://cs231n.github.io/convolutional-networks/>

range of 0.2-0.5 for reducing overfitting.<sup>8</sup> The model was trained using *nn.CrossEntropyLoss*, which is the standard loss function for multi-class classification problems. It compares predicted logits to true labels in a numerically stable way. The optimizer used was Adam with a learning rate of 0.001 and weight decay of 0.0001.<sup>9</sup> Adam was chosen because it adapts learning rates for each parameter and generally converges faster than traditional stochastic gradient descent. The weight decay value introduces L2 regularization, discouraging large weights and reducing overfitting. The training loop followed this common Pytorch cycle of clearing gradients, calculating loss, and updating: `optimizer.zero_grad() /// outputs = model(inputs) /// loss = criterion(outputs, labels) /// loss.backward() /// optimizer.step()`. Gradients were reset each iteration, predictions were generated through a forward pass, loss was computed, gradients were calculated using backpropagation, and weights were updated. Accuracy was calculated using `argmax(dim=1)` to determine predicted classes. Training and validation metrics were recorded after each epoch. During evaluation, *model.eval()* and *torch.no\_grad()* were used to disable dropout and gradient tracking, ensuring consistent inference behavior. During initial training, the model showed signs of overfitting, where training accuracy continued improving while validation accuracy plateaued. To address this, data augmentation was introduced using: (1) Random horizontal flipping, and (2) Random cropping with padding. These techniques are commonly used in CIFAR-10 training pipelines to improve generalization by exposing the model to varied versions of the data. I experimented with different dropout rates and weight decay values within commonly used ranges. Dropout values between 0.3 and 0.5 and weight decay values between 1e-4 and 1e-3 were tested to balance underfitting and overfitting.



After identifying the best configuration, the model was retrained from scratch to ensure consistent results. The final model achieved approximately 78% test accuracy, which aligns with expected performance for a simple CNN trained on CIFAR-10 without transfer learning. To further analyze performance, I examined misclassified images and generated a confusion matrix and classification report. These analyses revealed that the model struggled with visually similar classes, which is expected given the limited depth and capacity of the architecture.

*Methodology: ResNet-18 Transfer Learning with a Frozen Feature Extractor – Contributors: Yahir, Jennifer D.*

<sup>8</sup> PyTorch Developer Notes for Dropout –

<https://docs.pytorch.org/docs/2.11/generated/torch.nn.Dropout.html>

<sup>9</sup> PyTorch Adam Developer Notes - <https://docs.pytorch.org/docs/2.11/generated/torch.optim.Adam.html>

As motivation for this methodology, the frozen feature extractor can be understood as splitting the network into two parts: a fixed feature extraction stage and a trainable classification stage. The convolutional backbone of ResNet-18 transforms each image into a high-level feature representation, which is essentially a learned encoding of visual patterns. Because these layers are frozen, this representation remains constant throughout training. The only part that learns is the final layer, which takes these features and learns how to map them to CIFAR-10 class labels. This reduces the learning problem from “learning visual features and classification” to only “learning classification given string pretrained features.” While this greatly improves efficiency and stability, it also limits adaptability because the model cannot refine its internal representations for dataset-specific patterns.

In this experiment, we implemented transfer learning using a pretrained ResNet-18 model. Transfer learning means taking a model that has already learned useful visual features from a large dataset, such as ImageNet, and adapting it to a smaller target dataset. Instead of training every layer from scratch, this experiment used ResNet-18 as a frozen feature extractor and trained only a new final classification layer for CIFAR-10. This allowed us to test if general visual features learned from ImageNet could improve CIFAR-10 classification compared to the baseline CNN. First, we imported the same core libraries as in other notebooks: PyTorch, TorchVision, ResNet-18, image transforms, and sklearn/matplotlib. The notebook also reused the same reproducibility structure by setting a fixed seed and selecting CUDA when available. Keeping the same seed, batch size, validation split, and evaluation process consistent helped make this experiment comparable to the previous CNN baseline. The CIFAR-10 dataset was loaded using an 80/20 split. Because ResNet-18 was pretrained on ImageNet, the preprocessing pipeline had to be changed from the scratch CNN baseline. CNN-10 images were resized from 32x32 to 224x224, matching the expected input size for ResNet-style pretrained models. The images were also normalized using ImageNet mean and standard deviation values:  $mean=(0.485, 0.456, 0.406)$  and  $std=(0.229, 0.224, 0.225)$ . This was necessary because pretrained weights perform best when new inputs are processed similar to the data used during pretraining.

Next, DataLoaders were created for training, validation, and testing. The training loader used `shuffle=True` so batches appeared in a different order each epoch, while validation and test loaders used `shuffle=False` for evaluation. The batch size stayed at 128, matching the previous baseline to keep the training set up controlled. The model was then created by loading pretrained ResNet-18 weights. The original ResNet-18 final layer was designed for 1000 ImageNet classes, so it was replaced with a new `nn.Linear` layer outputting 10 logits for the 10 CIFAR-10 classes. After replacing the classifier, the convolutional backbone was frozen by setting `requires_grad = False` for the pretrained layers. Only the new final fully connected layer was left trainable. This meant the model was not relearning edges, textures, and object features from scratch; it was using pretrained visual representations and learning only how to map those representations to CIFAR-10 label. Training then followed the same PyTorch loop structure used earlier. For each batch, gradients were cleared with `optimizer.zero_grad()`, images were passed through the model, loss was computed using `CrossEntropyLoss`, gradients were calculated with

`loss.backward()`, and the optimizer updated only the trainable classifier weights. Since the frozen layers had `requires_grad=False`, they did not change during training. Accuracy was calculated with `argmax(dim=1)` by selecting the class with the highest logit. Validation was performed after each epoch using `model.eval()` and `torch.no_grad()`. This prevented gradient tracking and ensured the model was evaluated without training behavior. Training and validation loss/accuracy were recorded so model progress could be monitored and detect overfitting.

After training, the model was evaluated on the CIFAR-10 test set using `model.eval()` and `torch.no_grad()` to ensure proper inference behavior. Predictions were used to compute overall accuracy, a confusion matrix, and a classification report. This provided both overall performance and insight into class-level strengths and weaknesses.

*Methodology: ResNet-18 Transfer Learning with End-To-End Fine-Tuning – Contributors: Ido, Jennifer D., and Marcus F.*

For the third notebook, the project implemented a fine-tuned ResNet-18 transfer learning model for CIFAR-10 image classification. The purpose of this experiment was to test whether allowing part of a pretrained CNN to update during training would improve performance compared with both a CNN trained from scratch and a frozen transfer learning model. The notebook was structured around keeping the dataset, train/validation/test split, batch size, and evaluation metrics consistent with the earlier baselines, while changing the model training strategy for fine-tuning.

The initial training and evaluation loop structure was written by Ido and it followed the standard PyTorch workflow of training for one epoch, evaluation on validation data, and tracking loss and accuracy. I only modified and refined training configuration for fine-tuning, by changing the number of epochs in training, and Marcus contributed by changing the learning rate to better fit the behavior of a pretrained model. Specifically, the learning rate was reduced to  $1e-4$ , and the model was trained for 14 epochs. A lower learning rate was selected because fine-tuning pretrained weights requires more careful updates than training a randomly initialized mode; large updates can damage useful pretrained features instead of adapting them gradually.<sup>10</sup> He imported the libraries needed for dataset handling, model training, evaluation, visualization, and file organization. PyTorch was used for tensors, neural network layers, optimization, and GPU support. Torch vision was used to load CIFAR-10, apply image transformations, and import the pretrained ResNet-18 architecture. Additional libraries included `os` and `json` for organizing outputs, `random` and `NumPy` for reproducibility, `matplotlib` for learning curve plots, and `sklearn.metrics` for accuracy, confusion matrix, and classification reports.

The hyperparameters were defined as following: `SEED = 42`, `BATCH_SIZE = 128`, `LEARNING_RATE = 1e-4`, `NUM_EPOCHS = 14`, `VALIDATION_RATIO = 0.2`, and `NUM_CLASSES = 10`. The seed value was used to make the train/validation split and

---

<sup>10</sup> Pytorch Developer Notes for Transfer Learning for Computer Vision Tutorial – [https://docs.pytorch.org/tutorials/beginner/transfer\\_learning\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html)

initialization behavior more consistent across runs.<sup>11</sup> The batch size of 128 was kept consistent with earlier baselines so performance changes could be attributed more directly to the training strategy instead of batch-size difference. The validation ratio of 0.2 created an 80/20 split from the CIFAR-10 training set, allowing validation accuracy to be monitored during training. The learning rate of 1e-4 was a deliberate fine-tuning choice because pretrained layers should usually be updated more gently than randomly initialized layers.<sup>12</sup> The epoch count was set to 14 after adjustment, giving the model enough time to improve while avoiding unnecessary overtraining once validation performance began to plateau. A reproducibility function was then defined. It set the seed for Python's random, NumPy, PyTorch, and Cuda. It also sets: `torch.backends.cudnn.benchmark = False`. This was included because GPU operations and data splitting can introduce randomness, and reproducibility was important for comparing the fine-tuned model against earlier baselines.<sup>13</sup> The CIFAR-10 class names were explicitly defined as airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. This was not required for training itself, but it was important for interpretability. It allowed the confusion matrix, classification report, visualizations, and misclassification analysis to use meaningful class labels instead of numeric class IDs. CIFAR-10 has 10 fixed classes, so the final model layer was also designed to output 10 logits.<sup>14</sup> The preprocessing pipeline was changed specifically because this notebook used a pretrained ResNet-18. CIFAR-10 images are originally 32x32, but pretrained TorchVision ResNet-18 expects ImageNet-style preprocessing and larger image inputs. Therefore, the images were resized to 224x224 using `transforms.Resize((224, 224))`. This matched the expected input scale for the pretrained ResNet pipeline.<sup>15</sup> The notebook also normalized images using the ImageNet mean and standard deviation: `mean=(0.485, 0.456, 0.406)` and `std=(0.229, 0.224, 0.225)`. This was important because the pretrained ResNet-18 weights were learned under ImageNet preprocessing, so using the same normalization helped keep the input distribution compatible with the model's learned filters.<sup>16</sup>

The training transform included several augmentation steps: random horizontal flipping with probability 0.5, random cropping to 224 with padding of 4, color jitter with brightness, contrast, and saturation values of 0.2, and random rotation of 15 degrees. Random horizontal flipping was used because many CIFAR-10 objects remain recognizable when mirrored, and TorchVision's default flip probability is 0.5.<sup>17</sup> Random cropping with padding introduced small spatial shifts, helping the model become less sensitive to exact object placement. Color jitter

---

<sup>11</sup> Pytorch Developer Notes for Transfer Learning for Computer Vision Tutorial – [https://docs.pytorch.org/tutorials/beginner/transfer\\_learning\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html)

<sup>12</sup> Ibid.

<sup>13</sup> Ibid.

<sup>14</sup> PyTorch Torchvision library – <https://docs.pytorch.org/vision/stable/index.html>

<sup>15</sup> PyTorch ResNet18 library – <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>

<sup>16</sup> Ibid.

<sup>17</sup> Pytorch documentation on RandomHorizontalFlip – <https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomHorizontalFlip.html>

introduced moderate changes in brightness, contrast, and saturation, making the model less dependent on one exact color distribution. Random rotation introduced small orientation changes. Together, these augmentations were intended to reduce overfitting and improve generalization by making the training images appear slightly different each epoch.<sup>18</sup> The test transform was intentionally simpler. It resized images, converted them to tensors, and normalized them with ImageNet statistics, but it did not apply random augmentation. This separation was important because validation and test results should measure model performance on stable, clean data rather than on randomly altered images. The notebook loaded CIFAR-10 training and test sets using `torchvision.datasets.CIFAR10`, with `download=True` so the dataset would be automatically retrieved if it was not already available.<sup>19</sup>

The original training set was split into training and validation subsets using `random_split`, with the seed generator ensuring the sample split could be reproduced. The training subset was used for weight updates, while the validation subset was used after each epoch to check whether the model was improving on unseen data. This was important because fine-tuning can overfit quickly: a pretrained model already has strong features, so training accuracy may become very high even when validation improvement slows down. Dataloaders were then created for training, validation, and test sets. The training loader used `shuffle=True` so the model would see batches in a different order each epoch. The validation and test loaders used `shuffle=False` because their purpose was evaluation. The notebook also set `num_workers=2` to improve data loading efficiency while training. After creating the loaders, the notebook printed the batch shapes and confirmed that the input tensors had shape `[128, 3, 224, 224]`, matching the batch size, RGB channels, and resized ResNet input dimensions. He also visualized sample images. This step was useful as a sanity check, but it exposed a small visualization issue: the `imshow` helper function reused an older unnormalization formula intended for mean and standard deviation of 0.5, while the actual ResNet preprocessing used ImageNet mean and standard deviation. As a result, matplotlib produced clipping warnings because some displayed values were outside the expected image range. This did not affect model training, but it showed that the visualization helper should be updated if displaying ImageNet-normalized images. The model was then built using `torchvision.models.resnet18(pretrained=True)`. This loaded ResNet-18 pretrained on ImageNet. The notebook output showed a TorchVision warning that `pretrained=True` is deprecated and that the newer recommended approach is to use the weights augment instead. The behavior was still equivalent to loading ImageNet pretrained weights, but the warning identified a future improvement: replace `pretrained=True` with `weights=ResNet18_Weights.DEFAULT` or `ResNet18_Weights.IMAGENET1K_V1`.<sup>20</sup>

The final fully connected layer of ResNet-18 was replaced with `nn.Linear(model.fc.in_features, NUM_CLASSES)`. This was necessary because the original

---

<sup>18</sup> Transforming images, videos, boxes, and more <https://docs.pytorch.org/vision/main/transforms.html>

<sup>19</sup> Torchvision documentation – <https://docs.pytorch.org/vision/stable/index.html>

<sup>20</sup> Resnet18 documentation – <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html?>

ImageNet model predicts 1000 classes, while CIFAR-10 only has 10 classes. Replacing the final layer allowed the pretrained feature extractor to be reused while adapting the classifier head to the project's dataset.<sup>21</sup> The fine-tuning strategy was then applied. First, all model parameters were frozen by setting `param.requires_grad = False`. This prevented the entire pretrained network from updating. Then, the notebook unfroze `layer4` and the final fully connected layer by setting `requires_grad = True` for those parameters. This created a partial fine-tuning setup: earlier layers stayed frozen because they contain general low-level visual features such as edges, colors, and textures, while the final residual block and classifier were allowed to adapt to CIFAR-10's classes. This was a more flexible strategy than frozen feature extraction, but still safer and less computationally expensive than updating the entire network. The training and evaluation functions were then defined. The `train_one_epoch` function set the model to training mode with `model.train()`, looped through the training loader, moved images and labels to the device, cleared old gradients with `optimizer.zero_grad()`, computed outputs with `model(images)`, calculated the loss, called `loss.backward()` to compute gradients, and used `optimizer.step()` to update the trainable weights. Predictions were computed with `torch.argmax(outputs, dim=1)`, and `accuracy_score` was used to calculate epoch accuracy. This loop was part of the initial implementation credited to Ido, and it followed the standard PyTorch training pattern.<sup>22</sup> The `evaluate` function sets the model to evaluation mode with `model.eval()` and wrapped inference in `torch.no_grad()`. This prevented gradient calculation during validation and testing, making evaluation faster and ensuring the model did not update outside of training. The function returned loss, accuracy, labels, and predictions so later I could generate a confusion matrix, classification report, and per-class accuracy analysis. Also, to note, I corrected a line where predictions were detached and moved back to CPU before being converted to NumPy arrays, which helped avoid tensor/device issues during metric calculation.

The loss function used was `nn.CrossEntropyLoss(label_smoothing=0.1)`. Cross-entropy loss was appropriate because CIFAR-10 is a multi-class classification task. Label smoothing of 0.1 was added to reduce overconfidence by preventing the model from assigning all probability mass to one class during training. PyTorch's `CrossEntropyLoss` documentation supports `label_smoothing` as a parameter between 0 and 1, where 0 means no smoothing.<sup>23</sup> The optimizer used was `torch.optim.AdamW`, applied only to parameters with `requires_grad=True` through `filter(lambda p: p.requires_grad, model.parameters())`. This was important because frozen parameters should not be updated or included unnecessarily in optimization. AdamW was selected because it implements decoupled weight decay, meaning weight decay is handled separately from the adaptive gradient moments.<sup>24</sup> The optimizer used `lr=1e-4` and `weight_decay=1e-4`. The smaller learning rate supported stable fine-tuning, while weight decay

---

<sup>21</sup> Transfer Learning for Computer Vision

[https://docs.pytorch.org/tutorials/beginner/transfer\\_learning\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html)

<sup>22</sup> Computer vision pytorch one

<sup>23</sup> Cross entropy Loss –

<https://docs.pytorch.org/docs/2.11/generated/torch.nn.modules.loss.CrossEntropyLoss.html>

<sup>24</sup> AdamW – <https://docs.pytorch.org/docs/2.11/generated/torch.optim.AdamW.html>

added regularization to reduce overfitting. After training one epoch and then 14 epochs, plotting training and validation loss and accuracy curves was next. Then, to address overfitting, we implemented data augmentation. The transform was rewritten to resize, horizontal flip, random crop, color jitter, tensor conversion, and ImageNet normalization. The evaluation transform remained deterministic. The training and validation sets were re-split, and the DataLoaders were recreated to make sure the updated transforms were being used. The model and optimizer were then prepared for a new training run with the same fine-tuning structure. There were two retraining blocks total. The first retraining block showed improvement due to increased epochs, which will be discussed more in the result section. This suggests that the model continued improving slightly under the augmented fine-tuning setup. However, the model should have been reinitialized for a clean state, so a future cleanup should make sure the model is explicitly reloaded from pretrained weights before the final retraining run if the goal is a fully independent run.

*Methodology for Vision Transformer – Contributors: Jennifer D., Marcus F.*

For the ViT baseline, I used Akshay Gokhale's Medium article, *Training a Vision Transformer from Scratch on VIFAR-10*, as the main guide for building the model structure and training process. The guide explained the core ViT setup: splitting images into patches, embedding those patches, adding a CLS token and positional embeddings, passing the sequence through transformer encoder blocks, and using the CLS token for final classification. It also recommended a smaller ViT-Tiny-style architecture, AdamW, OneCycleLR, MixUp/CutMix, and strong regularization for training on a smaller dataset. The original plan was to use CIFAR-10. However, I ran into multiple loading issues. First, the CIFAR-10 download failed with a 503 Service Unavailable error. Then, when I tried to manually extract the [cifar-10-python.tar.gz](#) file with Python's tarfile module, the file could not be properly extracted because it was corrupted or incomplete. Since I was under a time restraint, I switched to MNIST. This let me still build and test the full Vision Transformer structure without spending the rest of the project time debugging the CIFAR-10 download.

The first coding step was importing the needed libraries. I used PyTorch for the model, training loop, tensors, and GPU support. I used torchvision for the dataset and transforms, NumPy for random values used in MixUp and CutMix, matplotlib for plotting, tqdm for progress bars, and sklearn for the confusion matrix and classification report. I also set a random seed so the dataset split and training process would be more consistent between runs. The device was set to CUDA if available otherwise CPY, because transformer models are more computationally expensive than simpler CNNs. Next, I created the configuration for the model. I kept the image size at 32x32 to stay close to the CIFAR-10 setup from the guide. Since MNIST images are grayscale instead of RGB, I changed in\_channels from 3 to 1. I used a patch size of 4, meaning each image was divided into small 4x4 patches. For a 32x32 image, this creates  $(32 / 4) \times (32 / 4) = 64$  patches. This matters because the transformer does not process the image as one full grid. Instead, it treats each patch like a token in a sequence. The guide explains that patching reduces

the sequence length from individual pixels to a much smaller set of patch tokens, making attention more practical.

After that, I loaded the MNIST dataset. I used transforms to resize the images to 32x32, convert them into tensors, and normalize them. Resizing was necessary because MNIST images normally come as 28x28, but my ViT was built around the 32x32 image size from the guide. Normalization helped scale the pixel values so the model could train more smoothly. I split the original training set into a training subset and validation subset using `random_split`. Then I used the same indices to create a clean validation dataset with the test transform instead of the training transform. This was important because the validation set should measure real performance, not performance on heavily augmented images. I then created the DataLoaders. The training Dataloader used `shuffle=True` so the model would see batches in a different order each epoch. The validation and test loaders used `shuffle=False` because their purpose was evaluation. I also used batch loading so the model could process many images at once instead of one image at a time.

I added MixUp and CutMix helper functions after that. Mixup creates a weighted blend of two images and also blends their labels. CutMix cuts a rectangular region from one image and pastes it into another image, then adjusts the label based on how much of each image is present. In the code, `one_hot()` converts integer labels like 3 or 7 into one-hot vectors so labels can be mixed mathematically. The `rand_bbox()` function calculates the random rectangle used for CutMix. The `apply_mixup_or_cutmix()` function randomly decides whether to use MixUp, CutMix, or no mixing. Since the labels become soft labels, I used `soft_target_cross_entropy()` instead of normal cross-entropy. This calculates loss by comparing the model's log probabilities to the mixed label distribution. Then I built the PatchEmbedding class. This was one of the most important parts of the ViT. Instead of manually slicing the image into patches, the code uses `nn.Conv2c` with `kernel_size=patch_size` and `stride=patch_size`. This means the convolution moves across the image in non-overlapping 4x4 blocks. The output channels are set to the embedding dimension, so each patch is immediately projected into a 192-dimensional vector. After the convolution, the shape is `[batch, embed_dim, grid_height, grid_width]`. The code then uses `flatten(2)` to collapse the spatial grid into a sequence of patches, and `transpose(1,2)` changes the shape to `[batch,num_patches, embed_dim]`, which is the format the transformer expects.

Now, this is fed to the multi-head self-attention block. The input shape is `[B, N, C]`, where B is batch size, N is the number of tokens, and c is the embedding dimension. Since the model has 64 patches plus 1 CLS token, N becomes 65. The code creates queries, keys, and values using one linear layer, `self.qkv = nn.Linear(embed_dim, embed_dim*3)`. Then it reshapes the result so the attention can be split across multiple heads. I used 3 heads, so each head gets part of the 192-dimensional embedding. The attention scores are calculated with `q @ k.transpose(-2,-1)`, which compares each token to every other token. The scaling factor prevents the dot products from becoming too large. Then softmax turns the scores into attention probabilities, and those probabilities are multiplied by v to create the final context vectors. I also had to debug this attention block. One error was accidentally using `==` instead of `=` when

defining `self.proj` and `self.proj_drop`. That made Python treat the line like a comparison instead of assigning a layer to the model. I fixed it by using proper assignment. This was important because the projection layer is what combines the output of the attention heads back into the original embedding dimension. After attention, I built the MLP and transformer encoder block. The MLP expands the embedding dimension by the `mlp_ratio`, applies GELU activation, uses dropout, and then projects the data back down to the original embedding size. The transformer block uses two main residual paths: one around self-attention and one around the MLP. The code uses PRe-Norm, meaning LayerNorm is applied before attention and before the MLP. This helps stabilize training because the input to each major operation is normalized before being transformed. I also fixed several errors here, including types like `supter` instead of `super`, `Dropout` instead of `Dropout`, and placing the `forward()` method in the wrong location. These errors showed how sensitive model code is to small syntax and indentation mistakes.

Once the blocks were finished, I built the full VisionTransformer class. First, the model calls the patch embedding layer. Then it creates a learnable CLS token using `nn.Parameter`. This CLS token is added to the beginning of every image's patch sequence and acts like a summary token. At the end of the transformer, the model uses this token to classify the whole image. The code also creates learnable positional embeddings. These are necessary because transformers do not naturally know the order of location of tokens. Without positional embeddings, the model would know what patches exist, but not where they came from in the image. The model then passes the sequence through a list of 12 transformer encoder blocks. I used `nn.ModuleList` so PyTorch would properly register every block as part of the model. After the transformer blocks, the output is normalized with LayerNorm. Then the model selects the CLS token output with `x[:, 0]` and passes it into a final linear classification head. Since MNIST has 10 digit classes, the final layer outputs 10 logits. After creating the model, I moved it to the selected device using `.to(device)`. I also calculated the total number of parameters and trainable parameters. This helped confirm that the model was similar to a ViT-Tiny setup, without about 5 million parameters. This size made sense because the guide recommended using a smaller ViT instead of a huge transformer, since smaller datasets can lead to overfitting.

For optimization, I used AdamW. I chose AdamW because it separates weight decay from the gradient update better than standard Adam, which helps with regularization. I also used OneCycleLR as the learning rate scheduler. This means the learning rate starts lower, increases up to a maximum, and then decreases again. The guide recommended this kind of scheduler because a fixed learning rate can make training less stable. Then I wrote an accuracy helper function. This function takes the model's logits and finds the predicted class using `argmax(dim=1)`. Since MixUp and CutMix create soft labels, the function also checks whether the labels are one-dimensional class IDs or multi-dimensional soft labels. If they are soft labels, it converts them back to hard labels with `argmax(dim=1)` before calculating accuracy. The training loop was the next major step. In `train_one_epoch()`, I set the model to training mode with `model.train()`, which activates dropout. Each batch is moved to the device, then MixUp or CutMix is applied. The optimizer gradients are reset with `optimizer.zero_grad()` so gradients

from the previous batch do not accumulate. The model produces logits, the soft-target loss is calculated, and `loss.backward()` computes gradients. I also used gradient clipping with `clip_grad_norm()` to prevent unstable updates, which can happen in transformer training. Then `optimizer.step()` updates the weights, and `scheduler.step()` updates the learning rate.

The validation loop used `model.eval()` and `torch.no_grad()`. This turns off dropout and prevents PyTorch from tracking gradients, making evaluation faster and more accurate. Validation did not use MixUp or CutMix because I wanted to test the model on clean images. The validation loss used normal cross-entropy because the labels were regular class labels again. After defining the loops, I trained the model for 30 epochs. I stored training loss, training accuracy, validation loss, and validation accuracy in a history dictionary so I could plot them later. I also saved the best model whenever the validation accuracy improved. This was important because the final epoch is not always the best epoch. Saving the best validation model helped make sure the final test evaluation used the strongest version of the model. After training, I plotted the loss and accuracy curves. The loss curve showed whether the model was improving over time, while the accuracy curve showed whether the model was learning meaningful patterns. Comparing training and validation curves also helped check for either overfitting or underfitting. If training accuracy kept rising while validation accuracy dropped, that would suggest the model was memorizing instead of generalizing. I also created prediction examples, a confusion matrix, and a classification report. The prediction examples visually checked whether the model's outputs matched the handwritten digits. Since MNIST is grayscale, I had to make sure the images were unnormalized correctly and displayed with `cmap='gray'`. The confusion matrix showed which digit classes were most often confused with each other, and the classification report gave precision, recall, and F1-score for each class. This gave a deeper evaluation than accuracy alone.

## **Results - Authors: Marcus Francois, Yahir Garcia , Jennifer Durnall**

The goal of this project was to evaluate how different deep learning approaches perform on image classification tasks while balancing computational cost and model accuracy. Specifically, three primary models were compared on the CIFAR-10 dataset: a CNN trained from scratch, a transfer learning model using a frozen ResNet-18 backbone, and a fine-tuned ResNet-18. Additionally, a Vision Transformer (ViT) was implemented on MNIST to demonstrate transformer-based performance. An ablation study was conducted by comparing the baseline CNN performance with and without data augmentation, isolating the effect of augmentation on model generalization. The results below analyze both quantitative metrics and qualitative behaviors across these approaches.

### *Evaluation Metrics*

The primary evaluation metric used in this project was test accuracy, which measures the percentage of correctly classified images on unseen data. This metric is appropriate for this task because CIFAR-10 is a balanced multi-class classification dataset, meaning each class is equally represented. In addition to accuracy, F1-score was used for the fine-tuned ResNet-18 and Vision Transformer models. F1-score provides a balance between precision and recall and is especially

useful for evaluating per-class performance and understanding how well the model handles misclassifications.

### *Results Comparison*

The table below summarizes the performance on each model:

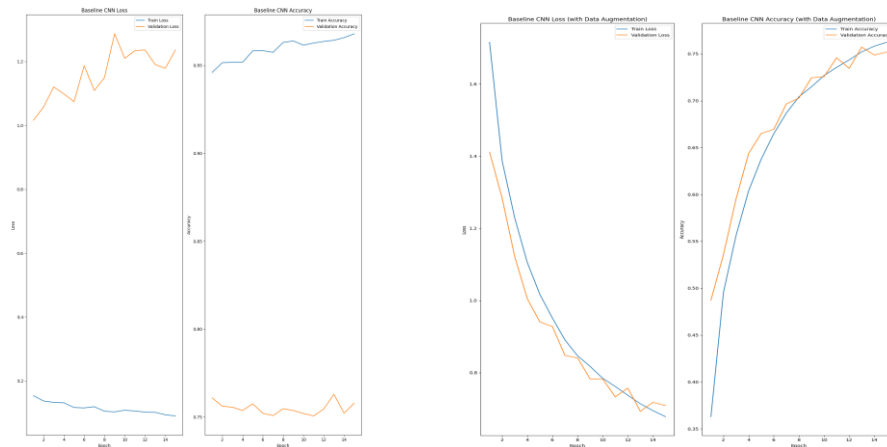
| Model                | Dataset  | Configuration                | F1Score | Test Accuracy |
|----------------------|----------|------------------------------|---------|---------------|
| CNN                  | CIFAR-10 | Baseline (No Augmentation)   | N/A     | 77.45%        |
| CNN                  | CIFAR-10 | Baseline (With Augmentation) | N/A     | 79.96%        |
| ResNet-18            | CIFAR-10 | Transfer Learning (Frozen)   | N/A     | 77.45%        |
| Fine-Tuned ResNet-18 | CIFAR-10 | Transfer Learning (Unfrozen) | 0.94    | 94.47%        |
| Vision Transformer   | MNIST    | Transformer Baseline         | 0.99    | 99.20%        |

The CNN trained from scratch achieved a test accuracy of 77.45% without augmentation and 79.96% with augmentation. This improvement demonstrates that data augmentation played a role in improving generalization. From the training curves below, the model without augmentation shows significant overfitting by the evidently large gap between validation and training curves. With augmentation, the validation accuracy improved and became more stable, suggesting that the model was exposed to more training examples.

However, the overall performance remained limited. This is expected because:

- The CNN had relatively shallow architecture
- It learned all features from scratch
- CIFAR-10 contains complex and visually similar classes

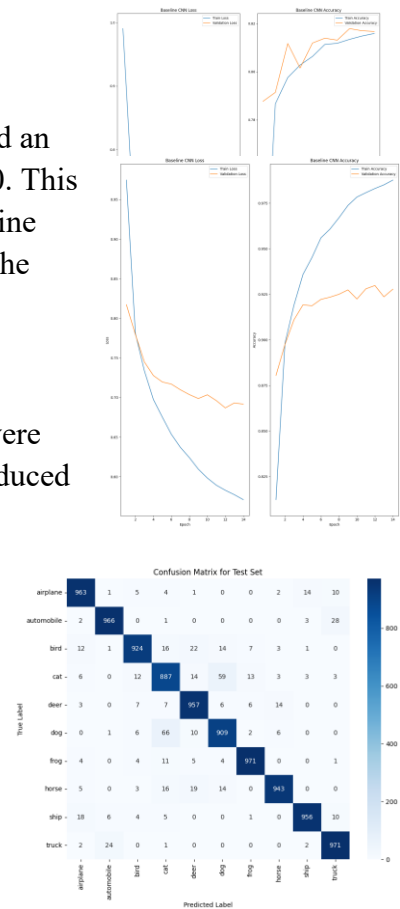
The frozen transfer learning model achieved a 77.45% accuracy, which is nearly identical to the baseline CNN without augmentation. This result is important because it shows that: (1) Simply using a pretrained model is not enough, and (2) If the feature extractor is frozen, the model cannot adapt to the new dataset. Even though ResNet-18 contains powerful pretrained features, those features were learned on ImageNet, which differs from CIFAR-10. Since only the final classification layer was trained, the model lacked the ability to refine internal representations. From the training curves, the frozen model converged quickly, but plateaued



early. This indicates that: the pretrained features were already useful, but the model reached its performance ceiling quickly due to lack of adaptability.

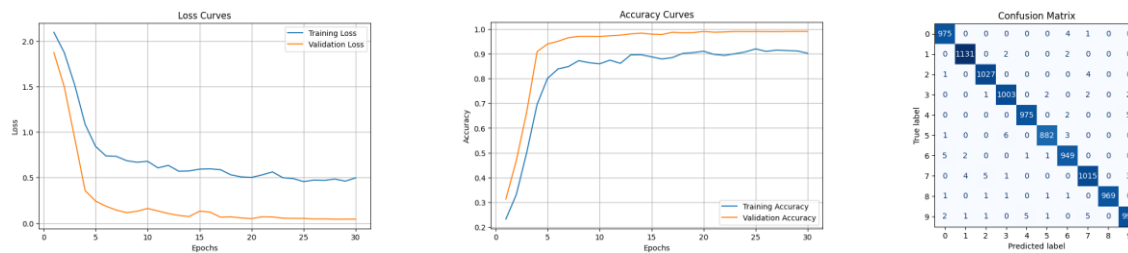
The fine-tuned ResNet-18 achieved 94.47% test accuracy and an F1 score of 0.94, making it the best-performing model on CIFAR-10. This represents a significant improvement (~15-17%) over both the baseline CNN and frozen transfer learning model. From the training curves, the fine-tuned model shows a some-what bump decrease in training and validation loss, continuous improvement in validation accuracy, and minimal overfitting. The key reason for this improvement is that the model was allowed to update its hidden layers, pretrained features were refined for CIFAR-10-specific patterns, and the learning rate was reduced to prevent destroying pretrained knowledge.

The confusion matrix for the most fine-tuned ResNet-18 model indicates strong performance on classes like frog, truck, automobile, and airplane, but lower performance on cat and dog. The most common confusion was when the true label was 'dog' but predicted as 'cat', and the second most was when the true label was 'cat' but the prediction was 'dog'. It also mistook 29 times 'truck' for 'automobile' and 24 times it mistook 'automobile' as 'truck'. Lastly, it also mistook 'ship' as 'airplane' 14 times and 'airplane' as 'ship' 18 times. This highlights an important limitation: Even high-performing CNNs struggle with fine-grained classification when classes are visually similar.



The Vision Transformer achieved a 99.20% accuracy and a 0.99% F1-score on MNIST. From the confusion matrix, the model showed near-perfect classification across all digits. As described in the methodology, the model successfully handled variation in handwriting and it accurately identified distinctive patterns in digits. However, MNIST is significantly simpler than

CIFAR-10 as it contains grayscale images with less variation and classes are easier to distinguish. Therefore, while the ViT performed extremely well, this does not directly prove it would outperform CNNs on CIFAR-10 without further testing.



## Discussion

Using the feature extract means splitting the network into two parts: a fixed feature extraction and a trainable classification stage. The ResNet-18 backbone simply turns each image into a set of visual patterns. Because these layers were frozen, they stayed the exact same during training. Only the final layer had to learn how to match those patterns to the right labels. This made training very efficient and stable in just 13 epochs, but the model couldn't adapt to specific patterns in the dataset, leading to an accuracy of 77.45%. Unfreezing the layers allowed the model to actually learn those specific patterns. While the unfrozen model reached convergence at 12 epochs, its capabilities bumped up accuracy to 94.47%.

Several important trends emerged across all experiments. For starters, data augmentation improved all models' ability to generalize. Adding augmentation improved CNN performance from 77.45% to 79.96%, showing that exposing the model to more diverse data reduces overfitting. From the frozen ResNet-18 experiment, it was also evident that pretrained features alone are not enough without updating the gradients in the hidden layers. With this, allowing deeper layers to update resulted in tremendous improvement in the transfer learning experiments; fine-tuning is significantly more powerful than feature extraction alone. Another key idea is that the model's capacity matters; the baseline CNN struggled due to limited depth and representational power, while ResNet-18 handled complex features more effectively. One important takeaway as well is that the dataset's complexity itself will strongly affect results. The Vision Transformer achieved extremely high accuracy on MNIST, but this reflects the simplicity of the dataset rather than superiority of the architecture alone.

If we had more time and less issues with loading the CIFAR-10 dataset, we would have liked to try the vision transformer on the dataset and see if it performed well. Because it is a smaller dataset, it would be interesting to see if the vision transformer effectively learns this smaller dataset or would possibly memorize even more than the CNN models. We would have

also liked to try more regularization techniques to smoothen the finetuned model's curves and reduce overfitting more.

## **Conclusion**

The main findings show that applying data augmentation to a custom built CNN improved accuracy from 77.45% to 79.97% on the CIFAR-10 dataset. When using transfer learning with the ResNet-18 model, keeping the model frozen resulted in a 77.45% accuracy. Unfreezing these layers allowed for the fine-tuned version to see a significant increase of 94.47% in accuracy and an F1-score of 0.94. Finally, the vision transformer trained on MNIST dataset achieved the highest overall performance of 99.20% in accuracy and a F1-score of 0.99.

## **Individual Contributions**

Jennifer: Implemented CNN from scratch and evaluation metrics, evaluation and implementation of fine-tuned CNN, implemented vision transformer

Marcus: Improving after training finetuned CNN and evaluation metrics, results analysis

Yahir: Designed and implemented initial ResNet-18 model

Ido: minimal design and implementation of fine-tuned CNN

## **AI Use**

In baseline 1, comments suggested by Gemini are left within the notebook—predominantly used for aid in design of researched ideas and debugging. In baseline 2, a link to his chat was left within the notebook for AI use. The AI use on Ido's end was lost, but AI was used to create confusion matrix and classification reports in addition to what he used it for.

## **References**

- CS231n. (n.d.). Convolutional neural networks. Stanford University.  
<https://cs231n.github.io/convolutional-networks/>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. arXiv. <https://arxiv.org/abs/1512.03385>
- PyTorch. (n.d.). Adam. PyTorch documentation.  
<https://docs.pytorch.org/docs/2.11/generated/torch.optim.Adam.html>
- PyTorch. (n.d.). AdamW. PyTorch documentation.  
<https://docs.pytorch.org/docs/2.11/generated/torch.optim.AdamW.html>
- PyTorch. (n.d.). CrossEntropyLoss. PyTorch documentation.  
<https://docs.pytorch.org/docs/2.11/generated/torch.nn.modules.loss.CrossEntropyLoss.html>
- PyTorch. (n.d.). Dropout. PyTorch documentation.  
<https://docs.pytorch.org/docs/2.11/generated/torch.nn.Dropout.html>
- PyTorch. (n.d.). Reproducibility. PyTorch documentation.  
<https://docs.pytorch.org/docs/2.11/notes/randomness.html>
- PyTorch. (n.d.). Training a classifier. PyTorch tutorials.  
[https://docs.pytorch.org/tutorials/beginner/blitz/cifar10\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html)
- PyTorch. (n.d.). Transfer learning for computer vision tutorial. PyTorch tutorials.  
[https://docs.pytorch.org/tutorials/beginner/transfer\\_learning\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html)

TorchVision. (n.d.). ResNet18. PyTorch documentation.  
<https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>

TorchVision. (n.d.). TorchVision documentation. PyTorch documentation.  
<https://docs.pytorch.org/vision/stable/index.html>

TorchVision. (n.d.). RandomHorizontalFlip. PyTorch documentation.  
<https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomHorizontalFlip.html>

TorchVision. (n.d.). Transforming images, videos, boxes, and more. PyTorch documentation.  
<https://docs.pytorch.org/vision/main/transforms.html>

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv. <https://arxiv.org/abs/1409.1556>

Gokhale, A. M. (n.d.). Training a Vision Transformer from scratch on CIFAR-10. Medium.  
<https://medium.com/@akshay.m.gokhale/training-a-vision-transformer-from-scratch-on-cifar-10-7804248048d9>